Customers, order details and applicable rules

Prompt: Fetch customers and their order details; make sure to apply appropriate rules.

Read loop: get_customers, then proposed get_orders using actual returned customer IDs; paginate as needed.

Prompt path · Actual business tool execution · Start / Resume · Interrupt / End

Start

Request body
organizationId + employeeId + userId + sessionId
New request: prompt + threadId when continuing
Resume: threadId + reviewId / proposalId + decisions

API service
Parse request
Check thread organization and session scope
Load matching checkpoint
New session: isolate previous history and Semantic context

New request ya resume?

New

LangGraph starts
State: request context + sessionId + threadId
prompt + current-session history

Resume

Checkpointing covers graph execution

Postgres Checkpointer
Automatic persistence after graph supersteps
Request IDs + sessionId + threadId + session history
Candidate batch + matches + versions + decisions
Tool results + pending review + frozen proposal

Restore matching session and thread state

Resume LangGraph from saved checkpoint
Same organizationId + sessionId + threadId

Node: prepareAndIntent - ONE LLM call
Input: prompt + current-session context
Output: polishedPrompt + intent

Intent?

Task / question

Node: loadRelevantMemories - three independent searches

Input: organizationId + sessionId + polishedPrompt
Search Semantic, Procedural and Episodic Memory in parallel

Three independently scoped queries

PostgreSQL Memory Store
Semantic: organizationId + sessionId
Procedural and Episodic: organizationId

Semantic results

Relevant Semantic Memory found?

Yes

semantic = matching facts / preferences
Only from the current session

No: semantic = []

Procedural results

Relevant Procedural Memory found?

Yes

procedural = matching instructions / rules

No: procedural = []

Episodic results

Relevant Episodic Memory found?

Yes

episodic = matching past episodes
Advisory steps and outcomes
Verify against the current request
Do not restore prior-session Semantic facts

No: episodic = []

Wait for all three searches
Save each result list in graph state
Any combination can contain memories

Memory remains in graph state

Available registered capabilities

Tool registry
Authorized tool names + descriptions
Schemas + effects + registered handlers

Authorized names and descriptions

Node: selectRelevantTools - LLM
Input: polishedPrompt + authorized tool descriptions
Output: selected tool names

Selected tool names

Initial planning

Registry lookup
Resolve selected names into actual tool definitions
Output: name + description + input schema + effect

Actual schemas

Assemble initial agent input
prompt + current-session history
tools: selected tool definitions
semantic: current-session facts or empty list
procedural: matching rules or empty list
episodic: verified hints or empty list
results: []
writeDeclined: false

Node: agentModel - LLM
Read initial assembled input
Decide initial answer or first tool calls

Approve matching proposalId

Output: response + calls list

Are there tool calls?

No: calls = []

Node: composeTaskResponse
Use agent response after calls are complete
Preserve original structured tool results

Node: recordEpisode
Input: prompt + performed steps + resolved ambiguity + outcome
LLM summarizes and saves Episodic Memory
Store request metadata and source sessionId

Yes: one or more calls

Node: authorizeToolCalls
Check selected names, access and input schemas
Resolve effects using registry metadata

Read or write?

Read

Node: executeRegisteredTools
Validate read arguments
Dispatch registered handlers by name

Arguments + request context

Execute frozen approved calls
Dispatch every approved call
Use a stable retry key for each batch entry

Approved arguments + request context

Output: response + calls list

Local PostgreSQL / registered tool backend
Organization-scoped queries or approved writes

Returned rows / write results

After tool execution

Append actual tool results to graph state
Each result: id + name + result
Avoid repeating unchanged completed reads

Rebuild input with updated results

Assemble agent input with results
prompt + current-session history
tools: existing selected tool definitions
semantic: current-session facts or empty list
procedural: matching rules or empty list
episodic: verified hints or empty list
results: updated actual tool results
writeDeclined: current value

Node: agentModel - LLM
Read actual tool results
Decide final answer or further tool calls
Treat empty tool results as valid information

Response: use /memory API
Submit all Procedural rules as one batch
Review all rule IDs together: approve / deny / edit

Completed result
status: completed
sessionId + threadId + response
Structured tool results or per-ID memory outcomes

Which review is pending?

Tool-write review

Tool-write decision?

Decline

After write decline

Set writeDeclined = true
Keep declined write unexecuted

Assemble agent input after decline
prompt + current-session history
tools: existing selected tool definitions
semantic: current-session facts or empty list
procedural: matching rules or empty list
episodic: verified hints or empty list
results: previous tool results or empty list
writeDeclined: true

Node: agentModel - LLM
Respect the declined write
Decide final answer or another permitted action
Do not re-propose the same declined write

Output: response + calls list

Save Procedural Memory

Save Semantic Memory

Semantic batch review

Semantic Memory addition

Node: validateSemanticDecisions
Validate reviewId, IDs, versions, text and targets
Collect decisions for the entire batch
Retain unchanged approvals and skips

Review batch ready?

All IDs decided

Edited text: affected facts only

Node: applyApprovedSemanticBatch - internal processing

Action for next decided candidateId?

Add new

Add new Semantic Memory
Exact approved text + stable retry key
Scope: organizationId + sessionId

Update existing

Update existing Semantic Memory
Only selected memoryIds in the same scope
Use the exact approved text

Skip

Skip candidate
No memory write

Collect per-ID outcome
candidateId + action + status + memoryIds

More IDs in the approved batch?

Yes: next submitted decision

No

Node: composeSemanticOutcome
Summarize saved, updated, skipped or failed items
Return outcomes for every candidateId

Node: polishSemanticBatch
Prepare all facts together
Assign stable candidateIds and item versions
Edits: reprocess affected facts
Preserve unchanged decisions

Node: checkSemanticToolScope
Check implied operations against registered tools
Identify limitations and eligible preferences

Node: searchSimilarSemanticBatch
Search all pending facts together
Scope: organizationId + sessionId
Return matching memoryIds per candidateId or empty lists

Node: buildSemanticReviewList
Combine proposed facts and existing matches
candidateId + exact proposed text + matching memoryIds
Exact create/update choices + item versions
Include limitations and eligible preference options

Missing decisions: remaining IDs

Interrupt: Semantic Memory review
Show the complete combined list
Per candidateId:
Add new / Update selected memoryIds / Skip
Or edit the proposed text

Write / mixed calls

Node: prepareWriteProposal
Fetch write schemas and required inputs
Gather inputs using registered read tools
Preview all targets, fields, values and effects
Validate and freeze every write call

Complete frozen proposal

Interrupt: await tool-write approval
proposalId + exact calls + complete previews

Pending result
status: approval_required
sessionId + threadId + review
reviewId / proposalId
Semantic review includes the combined candidate list
Graph remains paused

API response
HTTP 200: completed result
HTTP 202: pending review

End - current HTTP request

Complete original workflow retained. Highlighted boxes identify this scenario. Oversized vector page - zoom to read.